

Artificial Intelligence

Exam Answer Guide

15 questions with diagrams, tables and worked examples
plus supplements from the class notes

Contents

1. Q1. What is PEAS? PEAS descriptions of various systems
2. Q2. Problem formulation
3. Q3. Search algorithms
4. Q4. Types of agents
5. Q5. Properties of task environments
6. Q6. Supervised vs. unsupervised vs. reinforcement learning
7. Q7. Artificial Neural Networks, feed-forward networks and backpropagation
8. Q8. Hidden Markov Model (HMM)
9. Q9. Decision trees, entropy and pruning
10. Q10. Linear / polynomial regression, overfitting and underfitting
11. Q11. Support, confidence and lift using Apriori
12. Q12. Reinforcement learning (Q-learning, active vs. passive)
13. Q13. Naïve Bayes models and Support Vector Machines
14. Q14. Foundations of AI, Turing Test, and the four approaches
15. Q15. Architecture and components of a Fuzzy Logic System
16. Supplement A. Uniform Cost Search (UCS)
17. Supplement B. Adversarial search: Minimax and Alpha-Beta pruning
18. Supplement C. Reasoning under uncertainty
19. Supplement D. Machine learning basics and Responsible AI

Q1. What is PEAS? PEAS descriptions of various systems

PEAS is the framework used to specify the *task environment* of a rational agent before designing it. It stands for **P**erformance measure, **E**nvironment, **A**ctuators and **S**ensors.

- **Performance measure** – the criterion that defines how successful the agent is.
- **Environment** – the world in which the agent operates.
- **Actuators** – the means by which the agent acts on the environment.
- **Sensors** – the means by which the agent perceives the environment.

Agent	Performance measure	Environment	Actuators	Sensors
Vacuum cleaner	Amount of dirt cleaned, time taken, electricity used, noise, battery life	Rooms, floor, carpet, furniture, obstacles, stairs	Wheels, brushes, suction unit, dust bin	Dirt sensor, bump sensor, cliff sensor, location / camera sensor
Part-picking robot	Percentage of parts placed in the correct bin, speed, no damage to parts	Conveyor belt carrying parts, bins	Jointed arm and hand (gripper)	Camera, joint-angle sensors, touch / pressure sensor
Medical diagnosis system	Healthy patient, correct diagnosis, minimised cost and lawsuits	Patient, hospital, doctors, staff	Screen display of questions, tests, diagnoses, treatments, referrals	Keyboard entry of symptoms, findings, patient's answers, lab reports
Soccer match (robot player)	Goals scored, wins, team coordination, fewer fouls	Field, ball, teammates, opponents, referee	Legs (kick, run, turn), head, communication device	Camera, ball-position sensor, speed / direction sensor, communication receiver
Taxi driver (extra example)	Safe, fast, legal, comfortable trip, maximum profit	Roads, traffic, pedestrians, customers	Steering, accelerator, brake, signal, horn, display	Cameras, GPS, speedometer, odometer, sonar, keyboard

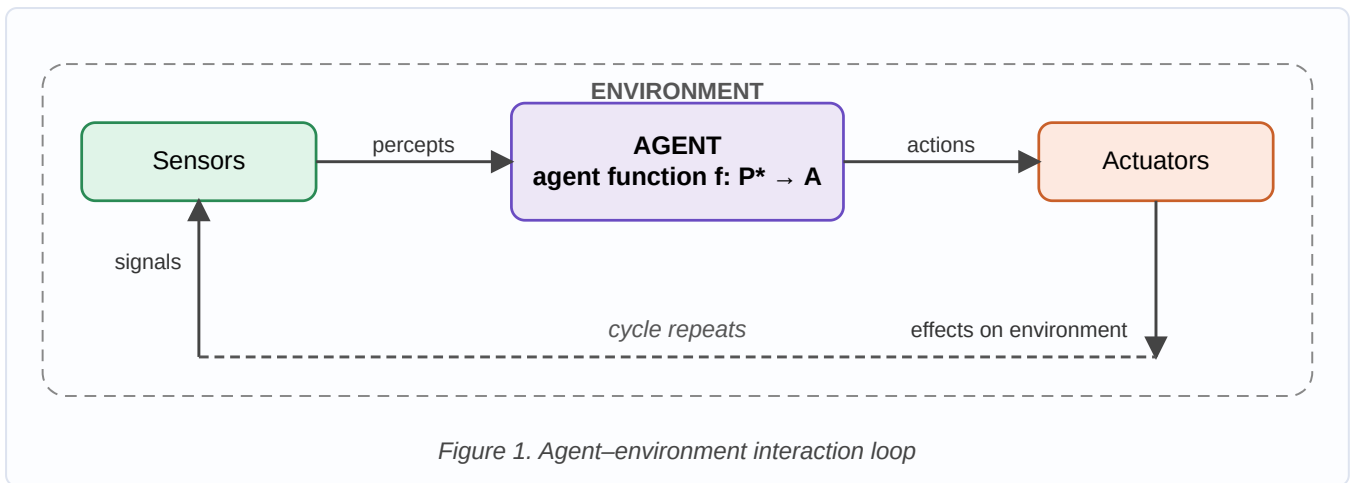
Why PEAS matters: the first step in designing any agent is to specify its task environment as fully as possible; a vague PEAS description leads to a badly designed agent.

More PEAS examples (from your notes):

Agent	Performance measure	Environment	Actuators	Sensors
Self-driving car	Safe arrival, speed, fuel efficiency	Roads, traffic, pedestrians	Steering, brakes, accelerator, signals	Cameras, GPS, LiDAR, radar
Chess-playing program	Wins games	Chessboard, opponent	Move selector on board	Board state (current positions)
Internet shopping agent	Price, quality, delivery	Online stores, suppliers	Order placement system	HTML / web scraper
Spam email filter	Block spam, allow genuine mails	Email inbox	Delete / label / move email	Email headers, content, sender info

Agent–environment loop: the environment generates signals → sensors convert them to percepts → the agent stores the percept sequence → the agent function $f: P^* \rightarrow A$ maps it to an action → actuators execute

the action and change the environment → the cycle repeats.



Q2. Problem formulation

A problem is formally defined by five components: **Initial state**, **Actions**, **Transition model** (RESULT(s, a)), **Goal test**, and **Path cost** (sum of step costs). The **state space** is the set of all states reachable from the initial state.

8-Queens problem (incremental formulation)

Component	Definition
States	Any arrangement of 0 to 8 queens on the board (the better version: arrangements of n queens, one per column in the leftmost n columns, with no queen attacking another)
Initial state	Empty board – no queens
Actions	Add a queen to any empty square (better: add a queen to the leftmost empty column so that it is not attacked)
Transition model	Returns the board with the queen added to the specified square
Goal test	8 queens are on the board and none is attacked
Path cost	Not relevant – only the final configuration matters (cost 1 per step if needed)

State-space size: about 1.8×10^{14} for the naive version versus only 2,057 for the improved one.

Vacuum world (2 locations)

Component	Definition
States	Agent location (A or B) $\times$ dirt status of each square $\rightarrow 2 \times 2^2 = \mathbf{8 \text{ states}}$
Initial state	Any state can be the initial state
Actions	Left, Right, Suck
Transition model	Left/Right move the agent (no effect at the edge); Suck removes dirt from the current square
Goal test	All squares are clean
Path cost	Each step costs 1, so path cost = number of steps

State notation used in class: **(Room A, Room B, Vacuum position)**, e.g. (D, D, A) = both rooms dirty, vacuum in A; (C, D, A) = A clean, B dirty; goal states (C, C, A) and (C, C, B). Sample transitions: from (D, D, A), Suck $\rightarrow$ (C, D, A) and Right $\rightarrow$ (D, D, B); from (C, D, B), Suck $\rightarrow$ (C, C, B) = goal.

8-Puzzle (3 $\times$ 3 sliding tiles)

Component	Definition
States	Location of each of the eight tiles and the blank – $9!/2 = \mathbf{181,440}$ reachable states
Initial state	Any state (a given scrambled arrangement)
Actions	Move the blank Left, Right, Up or Down (subset applicable depending on blank position)
Transition model	Returns the state resulting from the action (blank and neighbouring tile swap)
Goal test	Tiles are in the goal order, e.g. 1 2 3 / 4 5 6 / 7 8 blank

Component	Definition
Path cost	Each step costs 1; path cost = number of moves

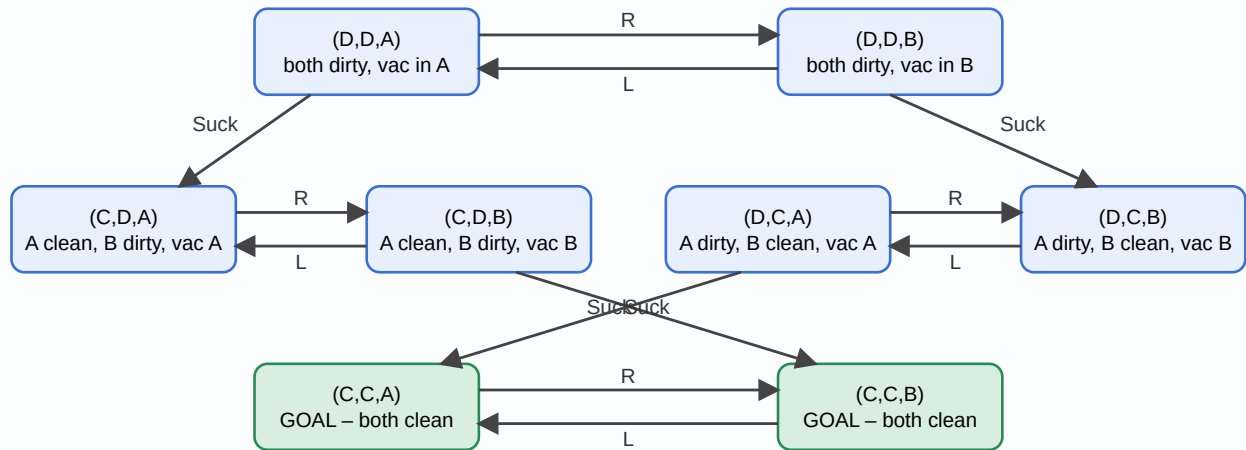


Figure 2. Vacuum-world state space: 8 states; actions Left (L), Right (R), Suck; self-loops omitted

Q3. Search algorithms

Search algorithms explore the state space from the initial state looking for a path to a goal. **Uninformed** (blind) strategies use only the problem definition; **informed** (heuristic) strategies use a heuristic $h(n)$ that estimates the cost from node n to the goal. Strategies are judged on completeness, optimality, time and space complexity (b = branching factor, d = depth of the shallowest goal, m = maximum depth, l = depth limit).

1. Breadth-First Search (BFS)

Expands the shallowest unexpanded node first, using a FIFO queue.

```
function BFS(problem):
  node <- Node(problem.INITIAL); if goal(node) return node
  frontier <- FIFO queue containing node
  explored <- empty set
  loop:
    if frontier is empty return failure
    node <- frontier.pop()           # shallowest node
    add node.state to explored
    for each action in problem.ACTIONS(node.state):
      child <- CHILD(problem, node, action)
      if child.state not in explored and not in frontier:
        if goal(child) return SOLUTION(child)
        frontier.insert(child)
```

Trace on a tree $S \rightarrow (A, B)$, $A \rightarrow (C, D)$, $B \rightarrow (E, G)$: frontier $S \rightarrow A, B \rightarrow B, C, D \rightarrow C, D, E, G \rightarrow$ goal G found. Visit order: **S, A, B, C, D, E, G**.

Second example (undirected graph with edges 1–2, 1–3, 1–5, 2–4, 3–4, 3–5): start at 1, visit its neighbours 2, 3, 5, then the remaining vertex 4 $\rightarrow$ BFS order **1-2-3-5-4**. BFS uses a queue and a visited list; applications are web crawling, social-network analysis, game AI and state-space search.

- Complete: yes (b finite). Optimal: yes when all step costs are equal.
- Time $O(b^d)$, space $O(b^d)$ – its major drawback is memory.

2. Depth-First Search (DFS)

Expands the deepest unexpanded node first, using a LIFO stack (or recursion).

```
function DFS(problem):
  frontier <- LIFO stack containing initial node
  explored <- empty set
  while frontier not empty:
    node <- frontier.pop()           # deepest node
    if goal(node) return SOLUTION(node)
    add node.state to explored
    for each child of node not in explored:
      frontier.push(child)
  return failure
```

Trace on the same tree: $S \rightarrow A \rightarrow C \rightarrow (\text{backtrack}) D \rightarrow (\text{backtrack}) B \rightarrow E \rightarrow G$. Visit order: **S, A, C, D, B, E, G**.

Notes example (tree $A \rightarrow B, C$; $B \rightarrow D, E$; $C \rightarrow F, G$): DFS goes A, B, D, backtracks to B and visits E, then C, F, G → order **A-B-D-E-C-F-G**. Applications: maze generation, puzzle solving, robot pathfinding.

- Complete: no in infinite or cyclic spaces (yes in finite spaces with a graph-search explored set). Optimal: no.
- Time $O(b^m)$, space only $O(b \cdot m)$ – its major advantage.

3. Iterative Deepening DFS (IDDFS / IDS)

Runs depth-limited DFS repeatedly with limit 0, 1, 2, ... combining the small memory of DFS with the completeness and optimality of BFS.

```
function ITERATIVE-DEEPENING-SEARCH(problem):
  for depth = 0 to infinity:
    result <- DEPTH-LIMITED-SEARCH(problem, depth)
    if result != cutoff return result

function DEPTH-LIMITED-SEARCH(problem, limit):
  return RECURSIVE-DLS(Node(problem.INITIAL), problem, limit)

function RECURSIVE-DLS(node, problem, limit):
  if goal(node) return SOLUTION(node)
  else if limit = 0 return cutoff
  cutoff_occurred <- false
  for each action in problem.ACTIONS(node.state):
    child <- CHILD(problem, node, action)
    result <- RECURSIVE-DLS(child, problem, limit - 1)
    if result = cutoff then cutoff_occurred <- true
    else if result != failure return result
  return cutoff if cutoff_occurred else failure
```

- Complete: yes. Optimal: yes (unit step cost). Time $O(b^d)$, space $O(b \cdot d)$.
- Repeated work on upper levels is small because most nodes lie at the bottom level.

4. Greedy Best-First Search

Expands the node that appears closest to the goal, using only the heuristic: **$f(n) = h(n)$** . Implemented with a priority queue ordered by h.

```
function GREEDY-BEST-FIRST(problem, h):
  frontier <- priority queue ordered by h(n), containing initial node
  explored <- empty set
  loop:
    if frontier empty return failure
    node <- frontier.pop()           # smallest h
    if goal(node) return SOLUTION(node)
    add node.state to explored
    for each child of node not in explored or frontier:
      frontier.insert(child)
```

- Complete: no (can get stuck in loops without an explored set). Optimal: no.

- Time and space $O(b^m)$ worst case, but a good heuristic improves this greatly.

Greedy best-first search expands the node with the lowest heuristic value, prioritising fast exploration over guaranteed optimal solutions; it is commonly used in navigation and pathfinding. Uniform-cost search, which orders by path cost $g(n)$ alone, is covered in Supplement A.

5. A* Search

Combines the cost so far and the estimated cost to go: $f(n) = g(n) + h(n)$, where $g(n)$ is the path cost from the start and $h(n)$ the heuristic estimate to the goal. Expands the node with the lowest $f(n)$.

```
function A-STAR(problem, h):
  frontier <- priority queue ordered by f = g + h, containing initial node
  explored <- empty set
  loop:
    if frontier empty return failure
    node <- frontier.pop()           # smallest f
    if goal(node) return SOLUTION(node)
    add node.state to explored
    for each action in problem.ACTIONS(node.state):
      child <- CHILD(problem, node, action)
      if child.state not in explored or frontier:
        frontier.insert(child)
      else if child is in frontier with higher g:
        replace that frontier node with child
```

Example: $S \rightarrow A$ (cost 1, $h(A)=3$), $S \rightarrow B$ (cost 4, $h(B)=1$), $A \rightarrow G$ (cost 5), $B \rightarrow G$ (cost 2); $h(S)=5$. $f(A)=1+3=4$, $f(B)=4+1=5 \rightarrow$ expand A: $f(G \text{ via } A)=6$; then expand B: $f(G \text{ via } B)=6$. Both give cost 6, and A* returns an optimal path.

- Complete: yes. Optimal: yes if h is **admissible** (never overestimates; tree search) and **consistent** ($h(n) \leq c(n,a,n') + h(n')$; graph search).
- Time and space are exponential in the worst case; memory is the practical limit.

Comparison

Algorithm	Complete	Optimal	Time	Space	Uses heuristic
BFS	Yes	Yes (equal costs)	$O(b^d)$	$O(b^d)$	No
DFS	No	No	$O(b^m)$	$O(b \cdot m)$	No
IDDFS	Yes	Yes (equal costs)	$O(b^d)$	$O(b \cdot d)$	No
Greedy BFS	No	No	$O(b^m)$	$O(b^m)$	Yes, $f = h$
A*	Yes	Yes (admissible h)	$O(b^d)$	$O(b^d)$	Yes, $f = g + h$

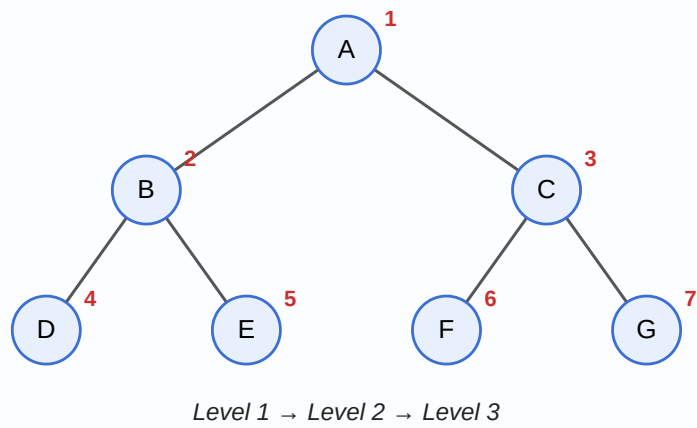


Figure 3a. BFS expands level by level (FIFO queue): A, B, C, D, E, F, G

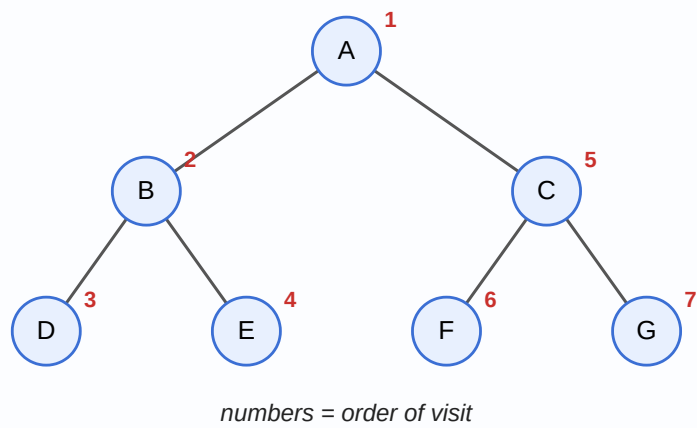
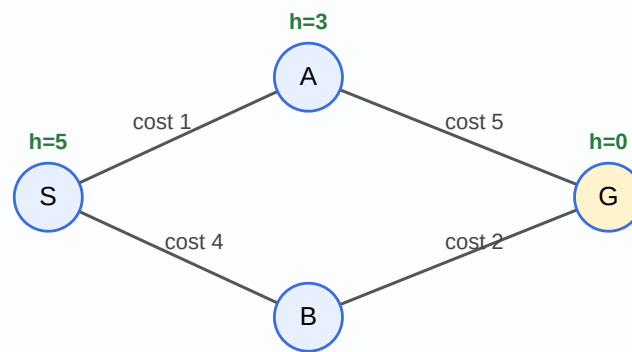


Figure 3b. DFS goes deep first and backtracks (LIFO stack): A, B, D, E, C, F, G



A^* : $f(A)=1+3=4$, $f(B)=4+1=5 \rightarrow$ expand A, then B; optimal cost 6

Figure 3c. Example graph for A^* ($f = g + h$) and greedy search ($f = h$)

Q4. Types of agents

An **agent** perceives its environment through sensors and acts on it through actuators. The agent function maps percept histories to actions. Five basic types, in increasing generality:

1. Simple reflex agent

Selects actions on the basis of the **current percept only**, ignoring history, using condition–action rules (*if condition then action*).

```
function SIMPLE-REFLEX-AGENT(percept):  
  state <- INTERPRET-INPUT(percept)  
  rule <- RULE-MATCH(state, rules)  
  return rule.ACTION
```

- Example: a thermostat; "if car-in-front-is-braking then initiate-braking".
- Works only if the environment is **fully observable**; can loop forever in partially observable worlds (a remedy is randomisation).

2. Model-based reflex agent

Maintains an **internal state** that depends on the percept history, to handle partial observability. It needs a **transition model** (how the world evolves and how its own actions affect it) and a **sensor model** (how the world appears in percepts).

```
function MODEL-BASED-REFLEX-AGENT(percept):  
  state <- UPDATE-STATE(state, last_action, percept, model)  
  rule <- RULE-MATCH(state, rules)  
  action <- rule.ACTION  
  return action
```

- Example: a self-driving car tracking other cars that are currently not visible.

3. Goal-based agent

Keeps track of the world state **and a set of goals**. It chooses actions that achieve the goal, using **search and planning** to consider the future ("what will happen if I do A?").

- More flexible than reflex agents: changing the goal does not require rewriting rules (e.g. a taxi with a different destination).
- Less efficient but adaptable. Cannot distinguish between a good and a better way of reaching the goal – only goal achieved / not achieved.

4. Utility-based agent

Uses a **utility function** that maps a state (or sequence of states) to a real number measuring the agent's "happiness". It picks the action with **maximum expected utility**, thus handling conflicting goals, trade-offs (speed vs. safety) and uncertainty.

- Example: a taxi choosing the route that is safer, quicker *and* cheaper rather than just any route reaching the destination.

5. Learning agent

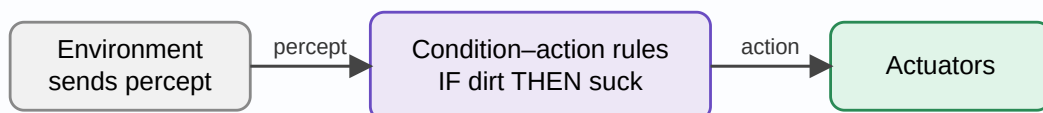
Can improve its performance with experience. Four conceptual components:

- **Learning element** – makes improvements by changing the performance element.
- **Performance element** – selects external actions (the agent types above).
- **Critic** – gives feedback on how well the agent is doing relative to a fixed performance standard.
- **Problem generator** – suggests exploratory actions leading to new, informative experiences.

Summary

Agent type	Uses	Handles	Limitation
Simple reflex	Condition–action rules, current percept	Fully observable, simple tasks	No memory, no foresight
Model-based	Internal state + world model	Partial observability	Still rule-driven
Goal-based	Goals + search/planning	Flexible goals	Only binary success/failure
Utility-based	Utility function, expected utility	Trade-offs, uncertainty	Needs good utility function and model
Learning	Critic + learning element + problem generator	Unknown environments, improves over time	More complex to build

Examples from your notes: simple reflex – thermostat, smoke alarm, basic Roomba (IF dirt THEN suck); model-based – robot vacuum keeping a room map, ABS braking; goal-based – GPS navigation, chess engine, delivery robot; utility-based – ride-booking app (cost vs. time vs. comfort), medical AI; learning – AlphaGo, Netflix recommendations, self-improving robots.



No internal state – reacts only to the current percept (thermostat, smoke alarm)

Figure 4a. Simple reflex agent

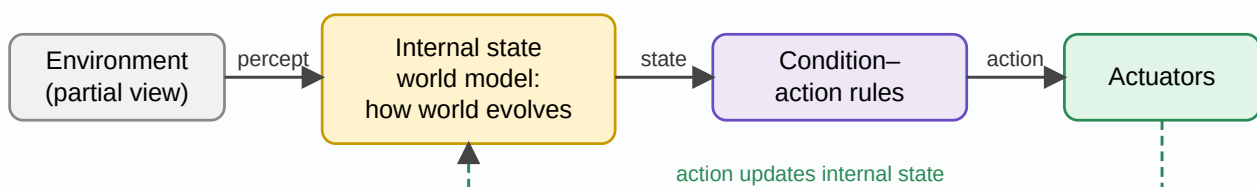


Figure 4b. Model-based reflex agent

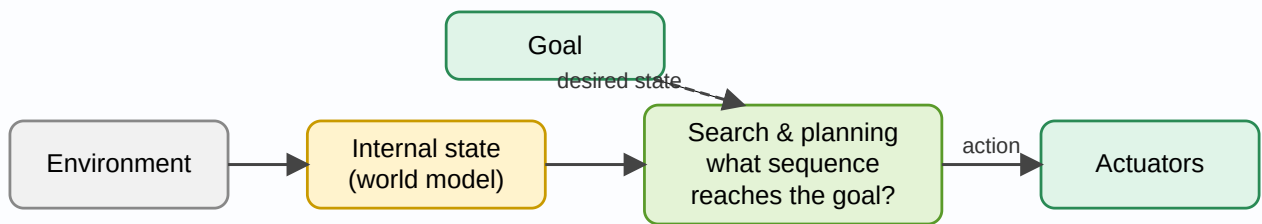


Figure 4c. Goal-based agent

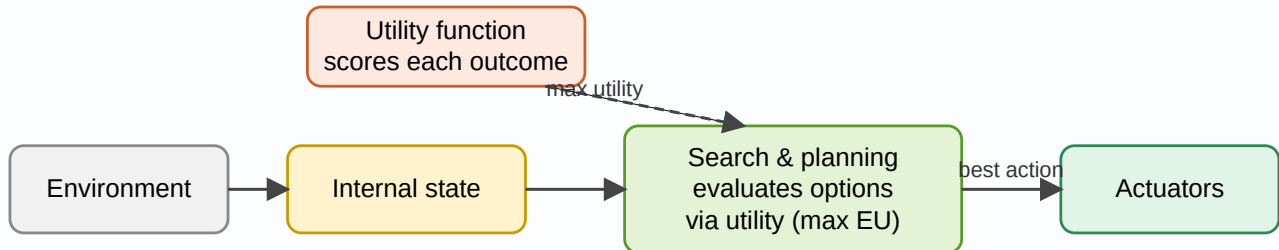


Figure 4d. Utility-based agent

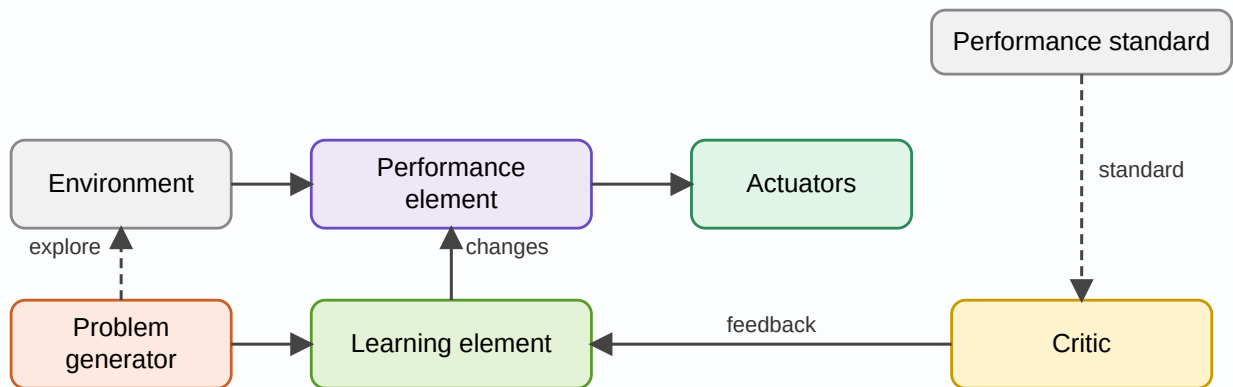


Figure 4e. Learning agent (performance element, critic, learning element, problem generator)

Q5. Properties of task environments

Property	Meaning	Easy case	Hard case
Fully vs. partially observable	Fully: sensors give access to the complete relevant state at each time. Partially: noisy, inaccurate sensors or missing information.	Fully – Chess, crossword, 8-puzzle	Partially – Poker, taxi driving, vacuum with only local dirt sensor
Deterministic vs. stochastic	Deterministic: next state is completely determined by current state and action. Stochastic: randomness / uncertainty in outcomes (if only other agents' actions are unpredictable it is <i>strategic</i>).	Deterministic – Chess, 8-puzzle	Stochastic – Taxi driving, poker, backgammon (dice)
Episodic vs. sequential	Episodic: experience is divided into atomic episodes; each decision is independent of previous ones. Sequential: current decision affects future decisions.	Episodic – spotting defective parts on an assembly line, image classification	Sequential – Chess, taxi driving, any planning task
Single vs. multiagent	Single: only one agent operates in the environment. Multiagent: other agents whose behaviour affects the performance measure (competitive or cooperative).	Single – crossword puzzle, solitaire	Multiagent – Chess (competitive), taxi driving (partly cooperative), soccer

Other properties

- **Static vs. dynamic** – the environment does not change while the agent is deliberating (static) vs. does (dynamic); *semi-dynamic* if only the score changes with time (chess with a clock).
- **Discrete vs. continuous** – finite number of states/percepts/actions (chess) vs. continuous values (taxi speed and position).
- **Known vs. unknown** – whether the agent knows the "laws" (outcomes of actions) of the environment.

Classification of examples

Task	Observable	Deterministic	Episodic	Static	Discrete	Agents
Crossword puzzle	Fully	Deterministic	Sequential	Static	Discrete	Single
Chess with a clock	Fully	Strategic	Sequential	Semi	Discrete	Multi
Poker	Partially	Stochastic	Sequential	Static	Discrete	Multi
Taxi driving	Partially	Stochastic	Sequential	Dynamic	Continuous	Multi
Medical diagnosis	Partially	Stochastic	Sequential	Dynamic	Continuous	Single
Part-picking robot	Partially	Stochastic	Episodic	Dynamic	Continuous	Single

The hardest case is **partially observable, stochastic, sequential, dynamic, continuous and multiagent** – e.g. taxi driving.

Notes examples: a chess game is fully observable, poker partially; tic-tac-toe is deterministic, rolling dice and stock trading are stochastic; an environment that is deterministic but has other agents is called **strategic**; a spam filter classifies each mail independently (episodic) while chess moves affect the whole

future (sequential); Sudoku is single-agent, chess is adversarial multiagent, a team of delivery drones is cooperative multiagent.

Q6. Supervised vs. unsupervised vs. reinforcement learning

Machine learning is the ability of a program to improve its performance on a task with experience. The type of learning depends on the feedback available.

Basis	Supervised	Unsupervised	Reinforcement
Data	Labelled input–output pairs (x, y)	Unlabelled inputs only	No fixed dataset; data generated by interacting with an environment
Feedback	Direct, correct answer for each example	None	Delayed reward / punishment signal
Goal	Learn mapping $f: X \rightarrow Y$ to predict outputs for new inputs	Discover hidden structure, patterns, groups	Learn a policy that maximises cumulative reward
Tasks	Classification, regression	Clustering, dimensionality reduction, association rules, anomaly detection	Control, game playing, robotics, resource allocation
Algorithms	Decision trees, Naive Bayes, SVM, linear regression, neural networks, k-NN	k-means, hierarchical clustering, PCA, Apriori	Q-learning, SARSA, policy gradients, Deep Q-Networks
Evaluation	Accuracy, precision/recall, MSE on test set	Cluster quality (silhouette), reconstruction error	Total reward obtained
Examples	Spam detection, house-price prediction, medical diagnosis	Customer segmentation, market-basket analysis	Self-driving, AlphaGo, robot walking
Key issue	Needs costly labelled data	Hard to validate results	Exploration vs. exploitation, credit assignment

A fourth category, **semi-supervised learning**, uses a small amount of labelled data with a large amount of unlabelled data.

Sub-types (from your notes): supervised = classification (logistic regression, decision tree, random forest, KNN, Naive Bayes, SVM) and regression (linear, polynomial, ridge, lasso); unsupervised = clustering (K-means, DBSCAN, mean-shift), dimensionality reduction (PCA, ICA) and association rule learning (Apriori, FP-growth, Eclat); reinforcement = Q-learning, SARSA, Deep Q-learning, with positive reinforcement (reward desired behaviour) and negative reinforcement (remove an unpleasant outcome after the right action). Reinforcement learning applies to gaming, robotics, autonomous vehicles, finance and recommendation.

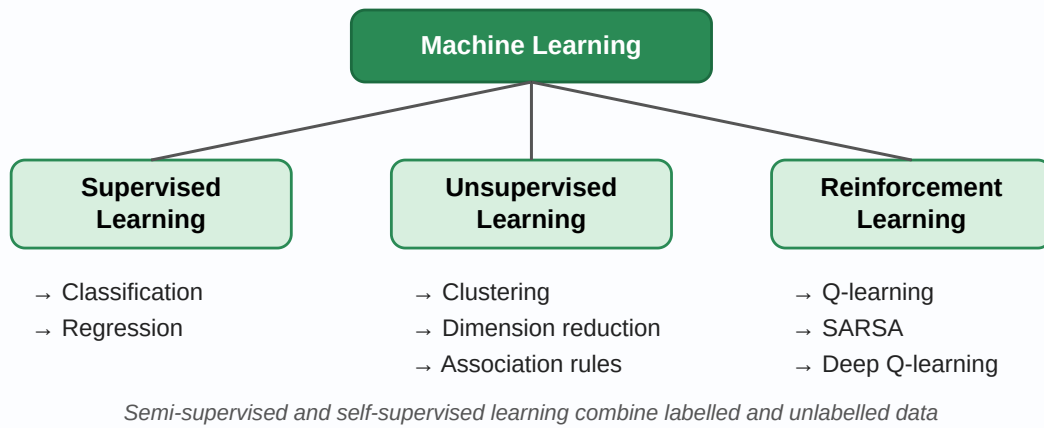


Figure 6a. Types of machine learning

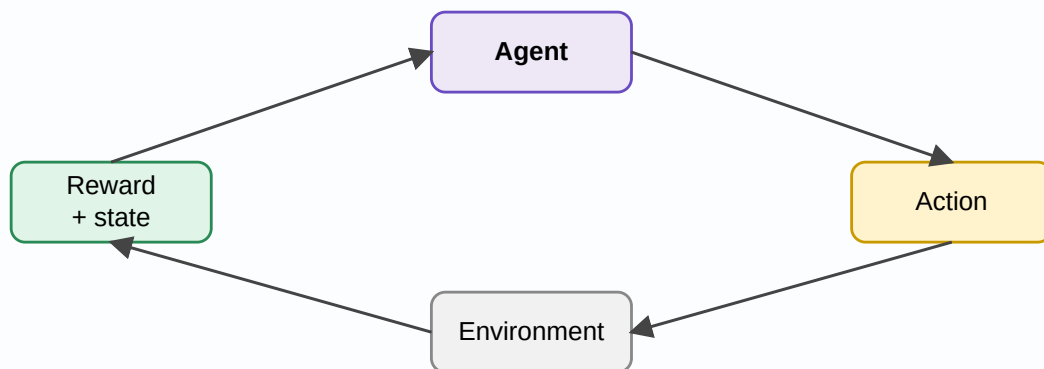


Figure 6b. Reinforcement-learning loop: action → environment → reward and new state → agent

Q7. Artificial Neural Networks, feed-forward networks and backpropagation

An **Artificial Neural Network (ANN)** is a computing model inspired by the brain: a network of simple processing units (**neurons**) joined by weighted links. Knowledge is stored in the weights, which are learned from data.

The artificial neuron

A unit computes a weighted sum of its inputs plus a bias and passes it through an **activation function** g :

$$a_j = g(\sum_i w_{ij} \cdot x_i + b_j)$$

Common activations: step / threshold, sigmoid $g(z) = 1 / (1 + e^{(-z)})$, tanh, ReLU $g(z) = \max(0, z)$.

Single-layer feed-forward network (perceptron)

- Inputs connect **directly to the output units**; no hidden layer. Information flows one way (no cycles).
- Perceptron learning rule: $w_i \leftarrow w_i + \alpha (y - \hat{y}) x_i$, where α is the learning rate.
- Can represent only **linearly separable** functions (AND, OR) – it cannot learn XOR (Minsky and Papert).

Multi-layer feed-forward network (MLP)

- Has one or more **hidden layers** between the input and output layers; each layer feeds only the next one (acyclic).
- A network with one hidden layer can approximate any continuous function (universal approximation); two layers solve XOR and other non-linear problems.
- Weights are learned by **backpropagation**.

Backpropagation algorithm

Backpropagation trains a multi-layer network by gradient descent on the error $E = \frac{1}{2} \sum (y - \hat{y})^2$, propagating the error backwards from the output layer to the hidden layers.

```
repeat until convergence:
  for each training example (x, y):
    1. FORWARD PASS: compute activations of every layer up to the output  $\hat{y}$ 
    2. Output error:  $\delta_k = g'(in_k) * (y_k - \hat{y}_k)$ 
    3. BACKWARD PASS:  $\delta_j = g'(in_j) * \sum_k (w_{jk} * \delta_k)$  for hidden
units
    4. Update weights:  $w_{ij} \leftarrow w_{ij} + \alpha * a_i * \delta_j$ 
```

- The weight update is $\Delta w = -\alpha \partial E / \partial w$ (gradient descent); the chain rule gives the deltas.
- Problems: local minima, slow convergence, vanishing gradients with sigmoid, overfitting (remedies: early stopping, regularisation, dropout, more data).

Applications

Image and speech recognition, handwriting recognition, language translation, forecasting, medical diagnosis, control systems.

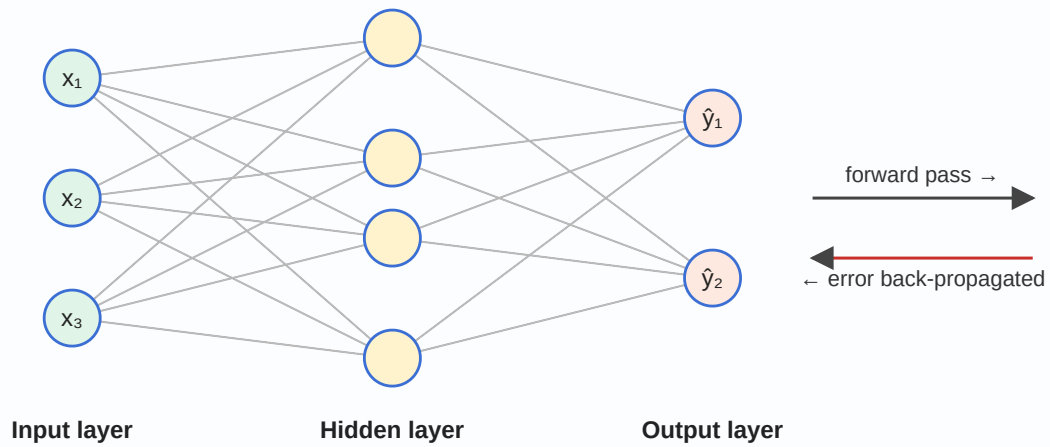


Figure 7. Multi-layer feed-forward network trained by back-propagation

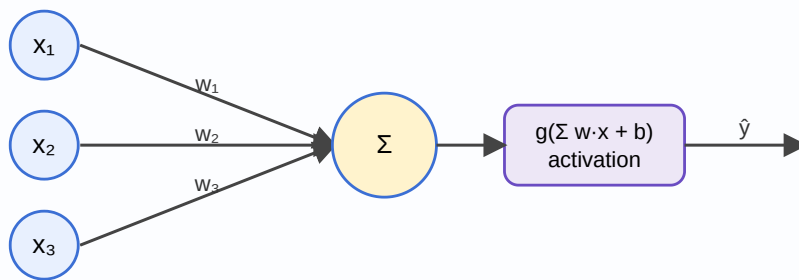


Figure 7b. A single neuron / perceptron: weighted sum, then activation function

Q8. Hidden Markov Model (HMM)

An **HMM** is a statistical model of a system that moves through a sequence of **hidden (unobservable) states**, each emitting an **observation** we can see. It is a temporal probabilistic model that rests on two assumptions:

- **Markov assumption** – the next state depends only on the current state: $P(X_t | X_1 \dots X_{t-1}) = P(X_t | X_{t-1})$.
- **Output independence (sensor) assumption** – an observation depends only on the current hidden state: $P(E_t | X_{1:t}, E_{1:t-1}) = P(E_t | X_t)$.

Components $\lambda = (A, B, \pi)$

Component	Meaning
States $S = \{s_1 \dots s_n\}$	Hidden states (e.g. weather: Rainy, Sunny)
Observations $O = \{o_1 \dots o_m\}$	Visible symbols (e.g. activity: walk, shop, clean)
Transition matrix A	$a_{ij} = P(X_t = s_j X_{t-1} = s_i)$; each row sums to 1
Emission (sensor) matrix B	$b_j(o) = P(E_t = o X_t = s_j)$
Initial distribution π	$\pi_i = P(X_1 = s_i)$

The joint probability of a state sequence and observations is

$$P(x_{1:T}, e_{1:T}) = \pi(x_1) \cdot b_{x_1}(e_1) \cdot \prod_{t=2..T} a(x_{t-1}, x_t) \cdot b_{x_t}(e_t)$$

Example

Hidden states: Rainy, Sunny. $\pi = (0.6, 0.4)$. Transitions: Rainy $\rightarrow$ Rainy 0.7, Rainy $\rightarrow$ Sunny 0.3, Sunny $\rightarrow$ Rainy 0.4, Sunny $\rightarrow$ Sunny 0.6. Emissions: Rainy $\rightarrow$ walk 0.1, shop 0.4, clean 0.5; Sunny $\rightarrow$ walk 0.6, shop 0.3, clean 0.1.

Probability of the state sequence (Rainy, Rainy) with observations (walk, shop) = $0.6 \times 0.1 \times 0.7 \times 0.4 = 0.0168$.

Three basic problems

Problem	Question	Algorithm
Evaluation (likelihood)	Given λ and an observation sequence, what is $P(\text{observations} \lambda)$?	Forward algorithm (dynamic programming, $O(N^2T)$)
Decoding	What is the most likely hidden state sequence?	Viterbi algorithm
Learning	Estimate A, B, π that maximise the likelihood of the observations	Baum–Welch (EM)

Temporal inference tasks: filtering $P(X_t | e_{1:t})$, prediction, smoothing $P(X_k | e_{1:t})$ for $k < t$, and most-likely explanation (Viterbi).

Forward algorithm

```

alpha_1(j) = pi_j * b_j(e_1)
alpha_t(j) = [ sum_i alpha_{t-1}(i) * a_ij ] * b_j(e_t)
P(e_1..e_T) = sum_j alpha_T(j)

```

Viterbi algorithm

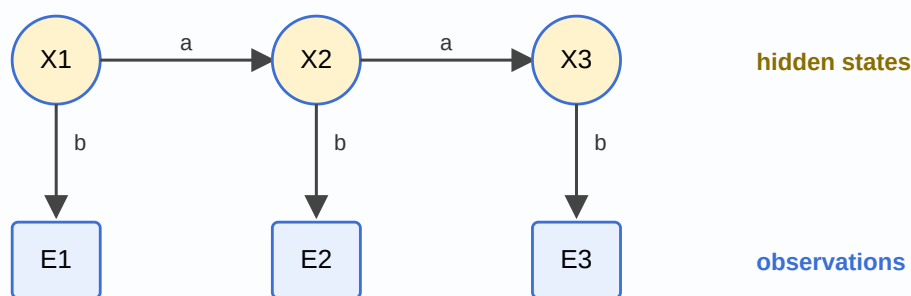
```

delta_1(j) = pi_j * b_j(e_1)
delta_t(j) = max_i [ delta_{t-1}(i) * a_ij ] * b_j(e_t)
psi_t(j)   = argmax_i [ delta_{t-1}(i) * a_ij ]      # back-pointer
backtrack from argmax_j delta_T(j) using psi to read off the best state path

```

Applications

Speech recognition, part-of-speech tagging, handwriting recognition, bioinformatics (gene finding), robot localisation, financial regime detection.



Transitions $A = P(X_t | X_{t-1})$, emissions $B = P(E_t | X_t)$, initial π

Figure 8. Hidden Markov model: hidden state chain emitting visible observations

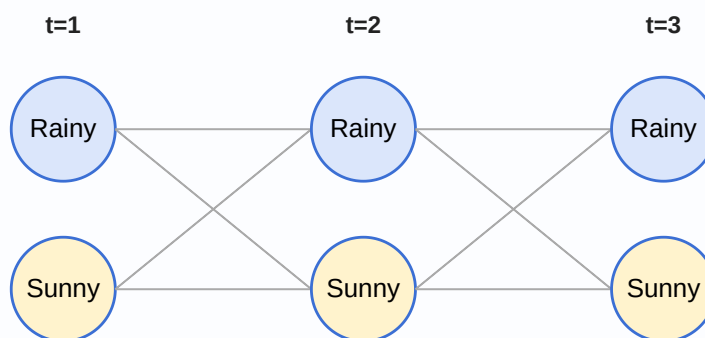


Figure 8b. Trellis: every state at time t connects to every state at $t+1$ (used by Forward and Viterbi)

Q9. Decision trees, entropy and pruning

A **decision tree** is a supervised model that classifies an example by asking a sequence of questions about its attributes. Each **internal node** tests an attribute, each **branch** is a test outcome, and each **leaf** gives a class (decision). Learning is a greedy, top-down, divide-and-conquer process (ID3 / C4.5 / CART).

```
function DECISION-TREE-LEARNING(examples, attributes, parent_examples):
  if examples is empty: return PLURALITY-VALUE(parent_examples)
  else if all examples have the same class: return that class
  else if attributes is empty: return PLURALITY-VALUE(examples)
  else:
    A <- attribute with the highest INFORMATION GAIN
    tree <- new decision tree with root test A
    for each value v of A:
      exs <- examples with A = v
      subtree <- DECISION-TREE-LEARNING(exs, attributes - A, examples)
      add branch (A = v) with subtree to tree
    return tree
```

Entropy and information gain

Entropy measures the impurity (uncertainty) of a set S with class proportions p_i :

$$H(S) = - \sum_i p_i \log_2 p_i \quad B(q) = -(q \log_2 q + (1-q) \log_2 (1-q))$$

Entropy is 0 for a pure set and 1 bit for a 50/50 boolean split. **Information gain** of attribute A is the expected reduction in entropy:

$$\text{Gain}(A) = H(S) - \sum_v (|S_v| / |S|) \cdot H(S_v)$$

Restaurant waiting problem (12 examples: 6 Wait = Yes, 6 Wait = No)

Entropy of the whole set: $H = B(6/12) = B(0.5) = 1$ bit.

Attribute Patrons (None / Some / Full):

Value	Examples	Yes	No	Entropy
None	2	0	2	0
Some	4	4	0	0
Full	6	2	4	$B(2/6) = 0.918$

Remainder = $(2/12)(0) + (4/12)(0) + (6/12)(0.918) = 0.459$. **Gain(Patrons) = 1 - 0.459 = 0.541 bits.**

Attribute Type (French, Italian, Thai, Burger): each value has an equal number of Yes and No (1/1, 1/1, 2/2, 2/2), so every subset has entropy 1 and **Gain(Type) = 0 bits.**

Patrons has the highest gain, so it becomes the root. The "None" and "Some" branches are already pure (No and Yes); the "Full" branch is split again on the next-best attribute (e.g. Hungry, then Type, Fri/Sat)

recursively.

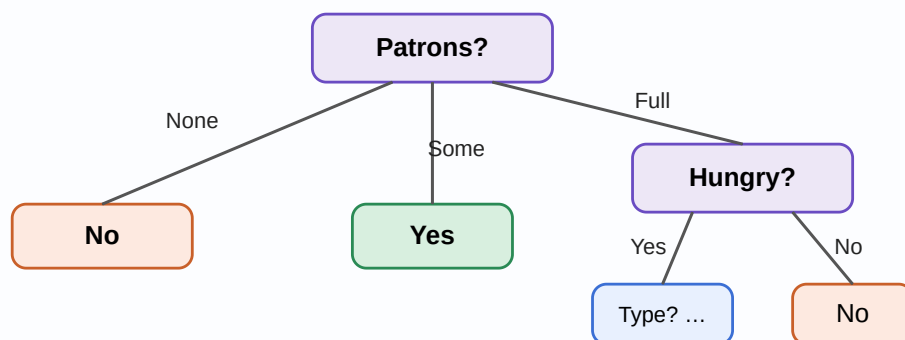
Pruning and overfitting

A fully grown tree fits noise in the training data and generalises poorly (overfitting). Pruning simplifies the tree:

- **Pre-pruning (early stopping)** – stop splitting when the gain is not statistically significant (e.g. chi-squared test) or when depth / node-size limits are reached.
- **Post-pruning** – grow the full tree, then replace subtrees with a leaf (majority class) if accuracy on a validation set does not drop (reduced-error pruning, cost-complexity pruning).
- Choose the tree size with **cross-validation**.

Advantages: easy to interpret, handles numeric and categorical data, little data preparation.

Disadvantages: unstable, prone to overfitting, biased towards attributes with many values (fixed by gain ratio).



Root = attribute with highest information gain (Patrons 0.541 bits, Type 0 bits)

Figure 9. Decision tree for the restaurant waiting problem (leaf = WillWait yes/no)

Q10. Linear / polynomial regression, overfitting and underfitting

Regression is supervised learning in which the output is a continuous value. We fit a function $h_w(x)$ to training pairs (x, y) by minimising the error.

Linear regression

Univariate model: $h_w(x) = w_1x + w_0$ (multivariate: $h_w(x) = w \cdot x = \sum w_i x_i$). The loss is the squared error (least squares):

$$\text{Loss}(w) = \sum_{j=1..N} (y_j - (w_1 x_j + w_0))^2$$

Closed-form solution for the line:

$$w_1 = (N \sum x_j y_j - \sum x_j \sum y_j) / (N \sum x_j^2 - (\sum x_j)^2) \quad w_0 = (\sum y_j - w_1 \sum x_j) / N$$

For many features, use **gradient descent**: $w \leftarrow w - \alpha \partial \text{Loss} / \partial w$ (repeat until convergence), or the normal equation $w = (X^T X)^{-1} X^T y$.

Polynomial regression

When data is non-linear, add powers of x as new features: $h_w(x) = w_0 + w_1x + w_2x^2 + \dots + w_dx^d$. The model is still *linear in the weights*, so the same least-squares machinery applies; the **degree d** controls model complexity.

Underfitting vs. overfitting

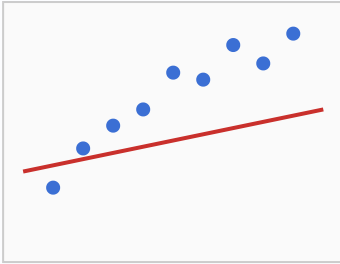
Aspect	Underfitting	Good fit	Overfitting
Model complexity	Too simple (e.g. a line on curved data)	Appropriate	Too complex (e.g. degree-15 polynomial on 16 points)
Training error	High	Low	Very low (near zero)
Test / validation error	High	Low	High
Problem	High bias , low variance	Balanced	High variance , low bias
Cause	Too few features / too simple model, too little training	–	Too many parameters, noise memorised, small dataset

Bias–variance trade-off: expected error = bias² + variance + irreducible noise. Increasing complexity lowers bias but raises variance.

Remedies for overfitting: more training data, simpler model, feature selection, **regularisation** (add $\lambda \sum w^2$ for ridge / L2 or $\lambda \sum |w|$ for lasso / L1 to the loss), cross-validation, early stopping, pruning / dropout.

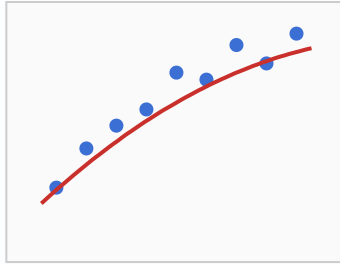
Remedies for underfitting: more expressive model (higher degree), more relevant features, less regularisation, train longer.

Underfitting (high bias)



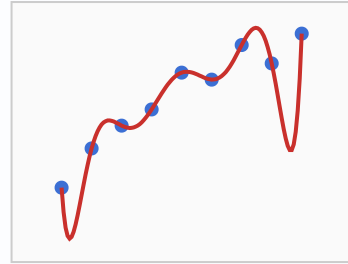
too simple: high train & test error

Good fit



balanced: low train & test error

Overfitting (high variance)



memorises noise: train=0, test high

Figure 10. Underfitting, good fit and overfitting on the same data

Q11. Support, confidence and lift using Apriori

Association rule mining finds rules $X \rightarrow Y$ in transaction data (market-basket analysis). Definitions, for N transactions:

$\text{Support}(X \rightarrow Y) = (\text{transactions containing } X \cup Y) / N$

$\text{Confidence}(X \rightarrow Y) = \text{Support}(X \cup Y) / \text{Support}(X)$

$\text{Lift}(X \rightarrow Y) = \text{Confidence}(X \rightarrow Y) / \text{Support}(Y)$

Lift > 1: positive association; lift = 1: independent; lift < 1: negative association.

Apriori principle: every subset of a frequent itemset must also be frequent (equivalently, if an itemset is infrequent, all its supersets are infrequent). This prunes the search space.

```
L1 <- frequent 1-itemsets (support >= min_sup)
for k = 2; L(k-1) not empty; k++:
  Ck <- join L(k-1) with itself, then prune candidates having an infrequent (k-1)-subset
  scan database, count support of each candidate in Ck
  Lk <- candidates in Ck with support >= min_sup
frequent itemsets = union of all Lk
generate rules X -> Y from each frequent itemset with confidence >= min_conf
```

Worked example

Transactions ($N = 5$), minimum support = 60% (count ≥ 3), minimum confidence = 70%.

TID	Items
T1	Bread, Milk
T2	Bread, Diaper, Beer, Eggs
T3	Milk, Diaper, Beer, Coke
T4	Bread, Milk, Diaper, Beer
T5	Bread, Milk, Diaper, Coke

Step 1 – 1-itemsets ($C_1 \rightarrow L_1$): Bread 4, Milk 4, Diaper 4, Beer 3, Coke 2, Eggs 1. Coke and Eggs fall below 3 and are removed. $L_1 = \{\text{Bread, Milk, Diaper, Beer}\}$.

Step 2 – 2-itemsets ($C_2 \rightarrow L_2$):

Itemset	Count	Frequent?
{Bread, Milk}	3	Yes
{Bread, Diaper}	3	Yes
{Bread, Beer}	2	No
{Milk, Diaper}	3	Yes
{Milk, Beer}	2	No
{Diaper, Beer}	3	Yes

Step 3 – 3-itemsets: the only candidate surviving the prune step is {Bread, Milk, Diaper} (all its 2-subsets are frequent); its count is 2 (T4, T5) < 3, so L3 is empty and the algorithm stops.

Step 4 – rules from the frequent pairs:

Rule	Support	Confidence	Lift	Verdict
Diaper → Beer	$3/5 = 0.60$	$3/4 = 0.75$	$0.75 / 0.60 = 1.25$	Strong, positive
Beer → Diaper	0.60	$3/3 = 1.00$	$1.00 / 0.80 = 1.25$	Strong, positive
Bread → Milk	0.60	$3/4 = 0.75$	$0.75 / 0.80 = 0.94$	Confident but lift < 1
Milk → Diaper	0.60	$3/4 = 0.75$	$0.75 / 0.80 = 0.94$	Confident but lift < 1

Interpretation: customers who buy Beer always buy Diapers (confidence 100%), and the two are bought together 25% more often than chance (lift 1.25). Bread → Milk looks confident but is not truly informative because lift is below 1.

If your question gives its own transaction table, follow exactly the same four steps with the stated minimum support and confidence.

Q12. Reinforcement learning (Q-learning, active vs. passive)

Reinforcement learning (RL) is learning what to do from rewards: an agent interacts with an environment, observes state s , takes action a , receives reward r and moves to state s' . The goal is to learn a **policy** $\pi(s)$ that maximises the expected discounted return $R = r_0 + \gamma r_1 + \gamma^2 r_2 + \dots$ ($0 \leq \gamma \leq 1$). The environment is modelled as a Markov Decision Process (MDP) with transition model $P(s'|s,a)$ and reward $R(s,a,s')$.

Key ideas: utility $U(s)$ of a state = expected discounted reward from s ; Bellman equation $U(s) = R(s) + \gamma \max_a \sum P(s'|s,a) U(s')$; and the **exploration vs. exploitation** dilemma.

Q-learning

Q-learning is a **model-free, off-policy, temporal-difference** method. It learns the action-value function $Q(s, a)$ = expected return of taking a in s and acting optimally afterwards, without knowing P or R in advance.

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a)]$$

```
initialise  $Q(s, a) = 0$  for all  $s, a$ 
repeat for each episode:
   $s \leftarrow$  start state
  repeat until  $s$  is terminal:
    choose  $a$  from  $s$  using epsilon-greedy policy (random with prob. epsilon, else
    argmax  $Q$ )
    take  $a$ , observe reward  $r$  and next state  $s'$ 
     $Q(s,a) \leftarrow Q(s,a) + \alpha * (r + \gamma * \max_{a'} Q(s',a') - Q(s,a))$ 
     $s \leftarrow s'$ 
```

Example: $Q(s,a) = 0$, $\alpha = 0.5$, $\gamma = 0.9$, $r = 10$, $\max Q(s', \cdot) = 20$. New $Q = 0 + 0.5 (10 + 0.9 \times 20 - 0) = 14$.

- Converges to the optimal Q^* if every state–action pair is visited infinitely often and α decays appropriately.
- The optimal policy is $\pi^*(s) = \operatorname{argmax}_a Q(s,a)$. SARSA is the on-policy variant, using the action actually taken next.

Passive vs. active reinforcement learning

Aspect	Passive RL	Active RL
Policy	Fixed policy π given	Agent must decide what to do (learn the policy)
Task	Policy evaluation: learn utility $U^\pi(s)$ of following π	Learn the optimal policy and utilities
Needs	Observing the environment (like a learner watching)	Exploration vs. exploitation; must consider the effect of actions
Methods	Direct utility estimation, Adaptive Dynamic Programming (ADP), Temporal-Difference (TD) learning: $U(s) \leftarrow U(s) + \alpha (r + \gamma U(s') - U(s))$	Active ADP (solve Bellman optimality), Q-learning, SARSA, exploration functions (ϵ -greedy, optimism under uncertainty)
Example	Estimating how good a given route is for a taxi	Learning the best route while still driving

A purely greedy active agent may get stuck in a sub-optimal policy; it must balance **exploitation** (use known best action) with **exploration** (try new actions to gain information).

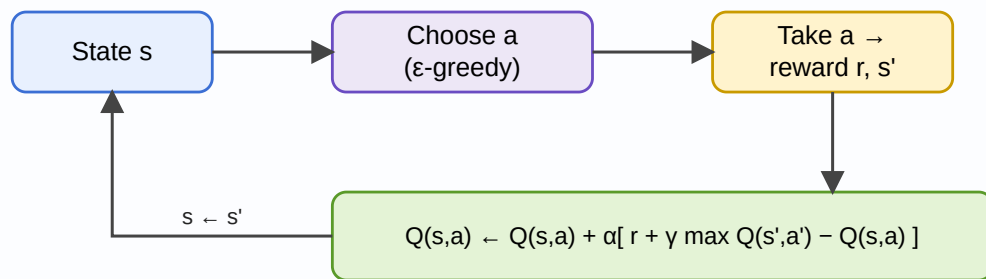


Figure 12. Q-learning cycle

Q13. Naïve Bayes models and Support Vector Machines

Naïve Bayes

A probabilistic classifier based on **Bayes' theorem** with the "naive" assumption that the features are **conditionally independent given the class**.

$$P(C \mid x_1 \dots x_n) = P(C) \cdot \prod_i P(x_i \mid C) / P(x_1 \dots x_n) \quad \hat{C} = \operatorname{argmax}_C P(C) \cdot \prod_i P(x_i \mid C)$$

- Parameters (class priors and per-feature likelihoods) are estimated by counting in the training data; the evidence in the denominator is the same for all classes and can be ignored.
- Laplace smoothing** avoids zero probabilities: $P(x|C) = (\text{count} + 1) / (N_C + k)$.
- Variants: Gaussian (continuous features), Multinomial (word counts), Bernoulli (binary features).

Mini example: 10 emails, 4 spam. 3 of the spam and 1 of the 6 non-spam contain "free". For a new mail containing "free": $P(\text{spam}) \cdot P(\text{free}|\text{spam}) = 0.4 \times 0.75 = 0.30$; $P(\text{not spam}) \cdot P(\text{free}|\text{not spam}) = 0.6 \times 0.167 = 0.10$. Normalised, $P(\text{spam} \mid \text{free}) = 0.30 / 0.40 = \mathbf{0.75}$, so the mail is classified as spam.

Advantages: fast, needs little data, works well for text and spam filtering, handles many features.

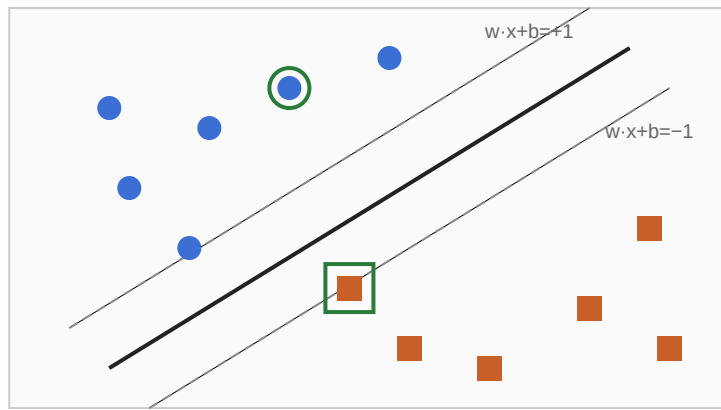
Disadvantages: independence assumption is rarely true; zero-frequency problem; poor probability estimates.

Support Vector Machine (SVM)

A supervised classifier (also usable for regression) that finds the **maximum-margin separating hyperplane** $w \cdot x + b = 0$ between two classes.

- Margin** = distance between the hyperplane and the closest points, $2/\|w\|$. The closest points are the **support vectors**; only they determine the boundary.
- Hard-margin optimisation (separable data): minimise $\frac{1}{2}\|w\|^2$ subject to $y_i(w \cdot x_i + b) \geq 1$ for all i .
- Soft margin** (noisy / overlapping data): minimise $\frac{1}{2}\|w\|^2 + C \sum \xi_i$ with slack variables $\xi_i \geq 0$; large C = narrow margin and few errors, small C = wide margin and more tolerance.
- Kernel trick:** for non-linearly separable data, map inputs to a higher-dimensional space implicitly through a kernel $K(x, x') = \phi(x) \cdot \phi(x')$ – linear, polynomial $(x \cdot x' + 1)^d$, RBF $\exp(-\gamma \|x - x'\|^2)$, sigmoid – and find a linear separator there.
- Decision rule: $\operatorname{sign}(\sum \alpha_i y_i K(x_i, x) + b)$, with nonzero α_i only for support vectors.

Aspect	Naïve Bayes	SVM
Type	Generative, probabilistic	Discriminative, geometric
Training	Counting, very fast	Quadratic optimisation, slower on large data
Assumption	Feature independence	Data separable (possibly after kernel mapping)
Output	Class probabilities	Class label (and distance from margin)
Best for	Text classification, small data	High-dimensional data, complex boundaries



hyperplane $w \cdot x + b = 0$; dashed lines: margin (width $2/\|w\|$); circled points: supp

Figure 13. SVM maximum-margin hyperplane

Q14. Foundations of AI, Turing Test, and the four approaches

Artificial Intelligence is the science and engineering of making intelligent machines – systems that perceive, reason, learn and act to achieve goals.

Foundations of AI (contributing disciplines)

Discipline	Contribution to AI
Philosophy	Logic, reasoning, mind as a physical system, rationality, knowledge and its sources
Mathematics	Formal logic, computability and algorithms, probability and statistics, decision theory
Economics	Utility, decision theory, game theory, Markov decision processes
Neuroscience	How the brain processes information; neurons, inspiration for neural networks
Psychology	Cognitive models, perception, learning, behaviour of humans and animals
Computer engineering	Fast hardware, memory and software to build AI systems
Control theory and cybernetics	Self-regulating systems that maximise an objective over time
Linguistics	Knowledge representation, grammar, natural language understanding

Four approaches to defining AI

	Human-like	Rational (ideal)
Thought / reasoning	Thinking humanly – cognitive modelling	Thinking rationally – "laws of thought"
Behaviour / action	Acting humanly – Turing Test	Acting rationally – rational agent

- **Thinking humanly (cognitive science):** build programs that think like humans, validated by psychological experiments and brain imaging (introspection, psychological experiments, brain imaging). Example: General Problem Solver (GPS).
- **Thinking rationally (laws of thought):** use logic (Aristotle's syllogisms) so that correct premises lead to correct conclusions. Problems: it is hard to express informal, uncertain knowledge in logic, and logical inference is computationally expensive.
- **Acting humanly (Turing Test):** see below.
- **Acting rationally (rational agent):** an agent acts to achieve the best expected outcome given its beliefs and the available information. This is the most general and widely used approach, because it does not require human-like behaviour, is mathematically well defined, and covers inference as one mechanism among others.

The Turing Test approach

Proposed by **Alan Turing (1950)** in "Computing Machinery and Intelligence". A human interrogator asks written questions to two hidden parties – a human and a computer. If the interrogator cannot tell which is the machine, the machine **passes** and is deemed intelligent. Typed communication avoids physical cues; the **Total Turing Test** also adds video (perception) and physical objects (manipulation).

Capabilities the computer needs:

- **Natural language processing** – to communicate in English.

- **Knowledge representation** – to store what it knows or hears.
- **Automated reasoning** – to answer questions and draw new conclusions.
- **Machine learning** – to adapt to new circumstances and detect patterns.
- **Computer vision** and **robotics** (Total Turing Test) – to perceive objects and move them.

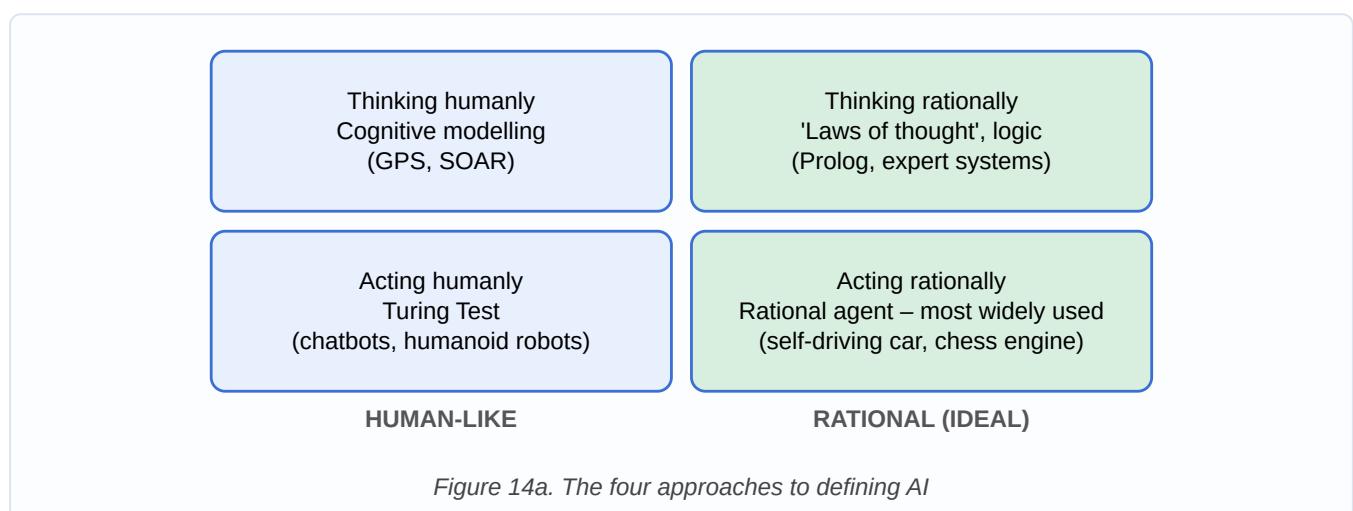
Criticisms: it tests imitation rather than real understanding (Searle's *Chinese Room* argument); it is anthropocentric; and engineers rarely pursue it – aeronautical engineering did not succeed by imitating pigeons, but by studying aerodynamics.

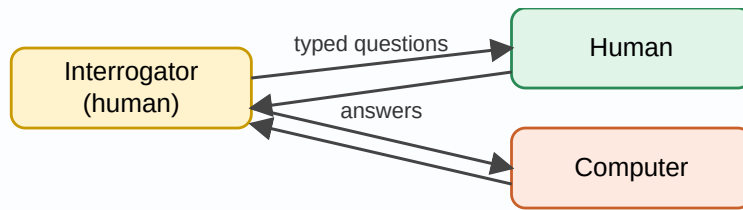
Rational agent vs. human thinking (from your notes)

A **rational agent** perceives through sensors, acts through actuators and always selects the action that maximises its expected performance measure, given its percept sequence and built-in knowledge. Rational does not mean omniscient: in a partially observable environment it may make a "wrong" choice yet still be rational for what it knew.

Aspect	Human thinking	Rational agent
Decision basis	Emotions, biases, intuition	Percepts + knowledge + logic
Consistency	Inconsistent under stress	Consistent
Memory	Limited (about 7 ± 2 chunks)	Can store millions of states
Speed	Slow for complex calculation	Very fast
Learning	Slow, needs repetition	Learns from vast data quickly
Errors	Cognitive biases (anchoring)	Only from bad data or model

Examples of the four approaches: thinking humanly – neural networks inspired by the brain, cognitive architectures such as SOAR; thinking rationally – Prolog programs, theorem provers, expert systems with IF-THEN rules; acting humanly – chatbots that pass as human in text conversation, humanoid robots; acting rationally – a self-driving car choosing the safest route, a chess engine choosing the best move. The term "Artificial Intelligence" was coined by John McCarthy at the 1956 Dartmouth Conference.





Passes if the interrogator cannot tell which is the machine

Figure 14b. The Turing Test set-up

Q15. Architecture and components of a Fuzzy Logic System

Fuzzy logic (Lotfi Zadeh, 1965) handles imprecise, vague information by allowing **degrees of truth** between 0 and 1 instead of only true/false. A fuzzy set A is described by a **membership function** $\mu_A(x) \in [0, 1]$; e.g. a temperature of 28 °C may be "warm" with $\mu = 0.7$ and "hot" with $\mu = 0.2$. Standard operations: AND = $\min(\mu_A, \mu_B)$, OR = $\max(\mu_A, \mu_B)$, NOT = $1 - \mu_A$.

Architecture (Mamdani fuzzy inference system)

Crisp input --> [1. FUZZIFICATION] --> fuzzy input --> [3. INFERENCE ENGINE] --> fuzzy output --> [4. DEFUZZIFICATION] --> Crisp output



Components

1. **Fuzzification module** – converts crisp sensor inputs (e.g. temperature = 28 °C) into fuzzy sets / membership degrees using the membership functions (triangular, trapezoidal, Gaussian).
2. **Knowledge base** – contains (a) the **database** of membership functions and linguistic variables (Cold, Warm, Hot), and (b) the **rule base**, a set of IF–THEN rules supplied by experts, e.g. *IF temperature is Hot AND humidity is High THEN fan speed is Fast*.
3. **Inference engine (decision-making unit)** – evaluates which rules fire and to what degree: it applies the fuzzy operators to the antecedents (min for AND), implies the consequent (clipping with min or scaling with product), and aggregates the outputs of all rules (max / union) into one fuzzy set.
4. **Defuzzification module** – converts the aggregated fuzzy output set back into a single crisp value for the actuator. Methods: **centroid (centre of gravity)** – most common, centre of area/bisector, mean of maxima, smallest/largest of maxima, weighted average.

Working example – air-conditioner fan

1. Crisp input: temperature = 28 °C → Warm 0.7, Hot 0.2.
2. Rules: IF Warm THEN Fan = Medium; IF Hot THEN Fan = Fast.
3. Inference: Medium is clipped at 0.7, Fast at 0.2; the two are combined using max.
4. Defuzzification: centroid of the combined area gives, say, fan speed ≈ 55 %.

Types and applications

- **Mamdani** (fuzzy consequents, defuzzification needed) vs. **Sugeno / Takagi–Sugeno** (crisp linear or constant consequents, weighted-average output – more efficient).
- Applications: washing machines, air conditioners, anti-lock brakes, camera autofocus, subway control, medical diagnosis, stock-trading systems.
- **Advantages:** handles uncertainty and vagueness, easy to design with human-like rules, tolerates imprecise input. **Disadvantages:** rule base can grow large, depends on expert tuning, no learning ability by itself (neuro-fuzzy systems add it).

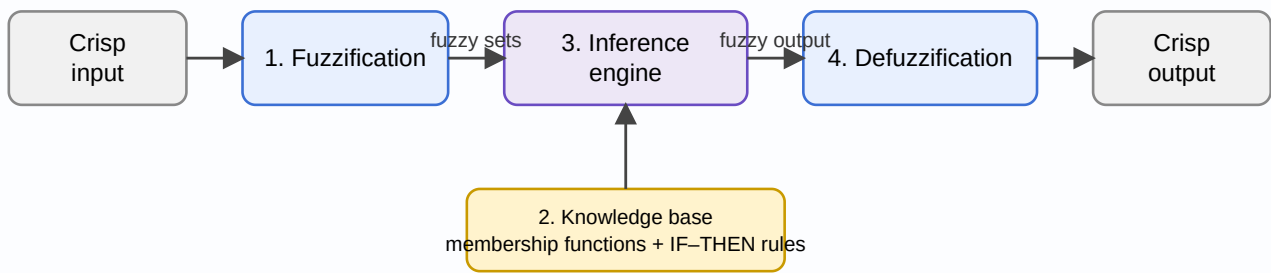


Figure 15a. Architecture of a fuzzy logic system (Mamdani)

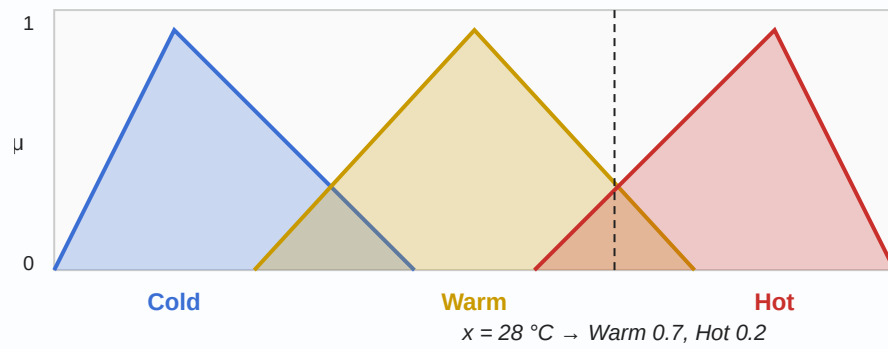


Figure 15b. Triangular membership functions for temperature

Supplement A. Uniform Cost Search (UCS)

The sections from here on come from the extra notes you attached; they sit outside the 15 questions but are useful for the same exam.

UCS is an uninformed search that expands the node with the **lowest cumulative path cost $g(n)$** , using a priority queue. It finds the cheapest path in a weighted graph when all edge costs are non-negative. With equal step costs it behaves like BFS.

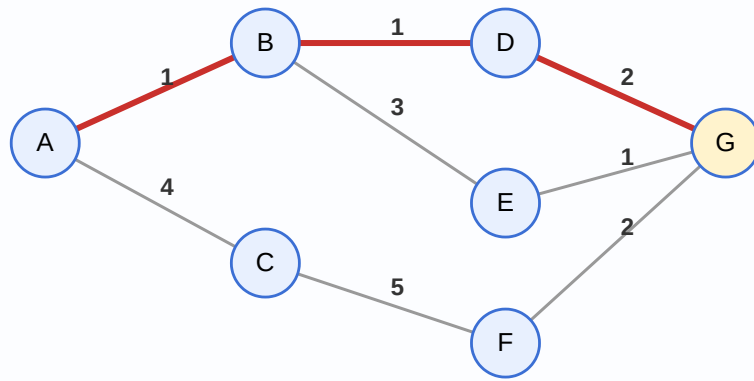
```
function UNIFORM-COST-SEARCH(problem):
  node <- Node(problem.INITIAL, cost = 0)
  frontier <- priority queue ordered by path cost, containing node
  explored <- empty set
  loop:
    if frontier is empty return failure
    node <- frontier.pop()           # lowest g(n)
    if goal(node) return SOLUTION(node) # goal test on EXPANSION, not generation
    add node.state to explored
    for each action in problem.ACTIONS(node.state):
      child <- CHILD(problem, node, action)
      if child.state not in explored or frontier: frontier.insert(child)
      else if child is in frontier with higher cost: replace it with child
```

Trace. Edges: $A \rightarrow B$ 1, $A \rightarrow C$ 4, $B \rightarrow D$ 1, $B \rightarrow E$ 3, $C \rightarrow F$ 5, $D \rightarrow G$ 2, $E \rightarrow G$ 1, $F \rightarrow G$ 2. Start A, goal G.

Step	Expand	Frontier after expansion (node : g)
1	A (0)	B:1, C:4
2	B (1)	D:2, C:4, E:4
3	D (2)	C:4, E:4, G:4
4	C (4)	E:4, G:4, F:9
5	E (4)	G:4 (the path via E costs 5, not better), F:9
6	G (4)	Goal reached

Least-cost path: **A → B → D → G, total cost 4.**

- Complete and optimal (non-negative costs). Time and space $O(b^{(1 + C^*/\epsilon)})$, where C^* is the optimal cost and ϵ the minimum step cost.
- Applications: route finding, network routing, sliding-tile puzzles, resource allocation.
- Drawbacks: explores many nodes in all directions, high memory, repeated updates when cheaper paths are found.



Least-cost path $A \rightarrow B \rightarrow D \rightarrow G$ (red), cost 4

Figure A. Uniform-cost search example graph

Supplement B. Adversarial search: Minimax and Alpha-Beta pruning

Adversarial search is used in competitive environments where agents have conflicting goals, typically **two-player zero-sum games** (what is good for one player is bad for the other; no win-win outcome) such as chess, checkers, Go and tic-tac-toe. Single-agent searches (BFS, DFS, A*) do not model an opponent, so Minimax and Alpha-Beta are used instead.

Minimax algorithm

A recursive, depth-first algorithm that gives the optimal move assuming **the opponent also plays optimally**. **MAX** picks the maximum value; **MIN** picks the minimum value. Values start at $-\infty$ for MAX and $+\infty$ for MIN.

```
function MINIMAX(node, depth, maximizingPlayer):
    if depth = 0 or node is terminal: return UTILITY(node)
    if maximizingPlayer:
        best <- -infinity
        for each child: best <- max(best, MINIMAX(child, depth-1, false))
        return best
    else:
        best <- +infinity
        for each child: best <- min(best, MINIMAX(child, depth-1, true))
        return best
```

Worked example. Tree: A (MAX) $\rightarrow$ B, C (MIN); B $\rightarrow$ D, E and C $\rightarrow$ F, G (MAX); leaves H=-1, I=4, J=2, K=6, L=-3, M=-5, N=0, O=7.

Level	Node values
MAX nodes D, E, F, G	D = $\max(-1, 4) = 4$; E = $\max(2, 6) = 6$; F = $\max(-3, -5) = -3$; G = $\max(0, 7) = 7$
MIN nodes B, C	B = $\min(4, 6) = 4$; C = $\min(-3, 7) = -3$
MAX root A	A = $\max(4, -3) = 4$ (best path A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ I)

- Complete (finite tree); optimal against an optimal opponent. Time $O(b^m)$, space $O(b \cdot m)$ (b = branching factor, m = maximum depth).
- Limitation: far too slow for chess or Go because of the huge branching factor – improved by alpha-beta pruning.

Alpha-Beta pruning

An optimisation of minimax that returns **the same move** but skips branches that cannot affect the decision.

- α = best (highest) value found so far for MAX along the path; initial $\alpha = -\infty$.
- β = best (lowest) value found so far for MIN along the path; initial $\beta = +\infty$.
- **Prune the remaining children whenever $\alpha \geq \beta$.**
- MAX nodes update only α ; MIN nodes update only β ; node values (not α/β) are passed up, while α and β are passed down to children.

```

function ALPHA-BETA(node, depth, alpha, beta, maximizingPlayer):
  if depth = 0 or node is terminal: return UTILITY(node)
  if maximizingPlayer:
    value <- -infinity
    for each child:
      value <- max(value, ALPHA-BETA(child, depth-1, alpha, beta, false))
      alpha <- max(alpha, value)
      if alpha >= beta: break          # beta cut-off
    return value
  else:
    value <- +infinity
    for each child:
      value <- min(value, ALPHA-BETA(child, depth-1, alpha, beta, true))
      beta <- min(beta, value)
      if alpha >= beta: break          # alpha cut-off
    return value

```

Worked example. A (MAX) $\rightarrow$ B, C (MIN); B $\rightarrow$ D, E; C $\rightarrow$ F, G (all MAX); leaves D: 2, 3; E: 5, 9; F: 0, 1; G: 7, 5.

1. A, B and D start with $\alpha = -\infty$, $\beta = +\infty$. D = max(2, 3) = **3**.
2. B (MIN) takes $\beta = \min(+\infty, 3) = 3$.
3. E receives $\alpha = -\infty$, $\beta = 3$. First leaf 5 gives $\alpha = 5 \geq \beta = 3$, so the leaf **9 is pruned**; E = 5. B = min(3, 5) = **3**.
4. A (MAX) sets $\alpha = 3$ and passes $\alpha = 3$, $\beta = +\infty$ to C and F. F = max(0, 1) = 1.
5. C (MIN) sets $\beta = \min(+\infty, 1) = 1$; now $\alpha = 3 \geq \beta = 1$, so **subtree G (leaves 7, 5) is pruned**; C = 1.
6. A = max(3, 1) = **3**.

Move ordering

Ordering	Effect	Time complexity
Worst	Best moves examined last; no pruning, same as minimax	$O(b^m)$
Ideal	Best moves examined first; most branches pruned, searches about twice as deep in the same time	$O(b^{(m/2)})$

Heuristics for good ordering: examine the best moves first, use domain knowledge (chess: captures, then threats, then forward moves, then backward moves) and remember repeated states.

Applications: chess engines (Stockfish), checkers, tic-tac-toe, and adversarial decision making in security, economic modelling and robotics.

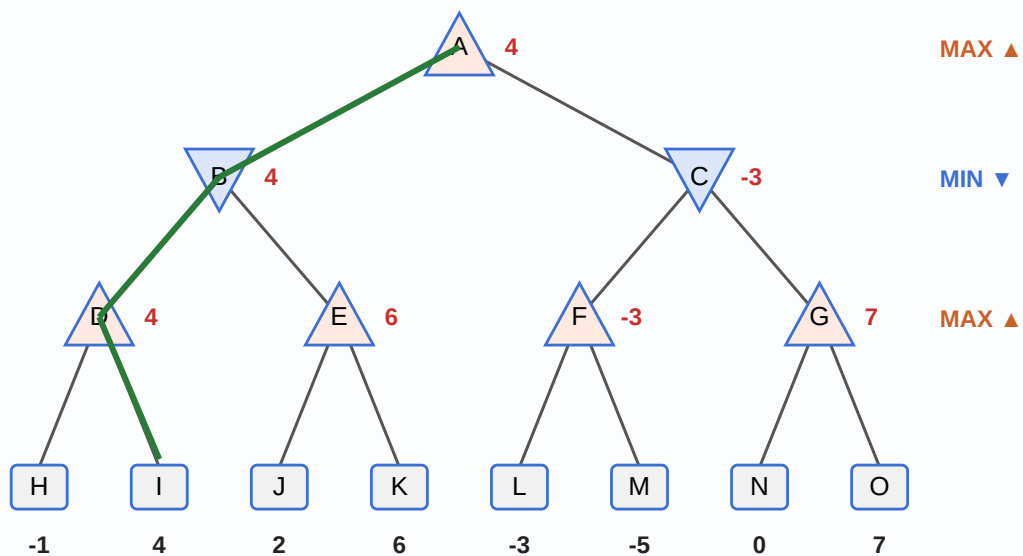


Figure B1. Minimax tree: red numbers are backed-up values; best path A → B → D → I (value 4)

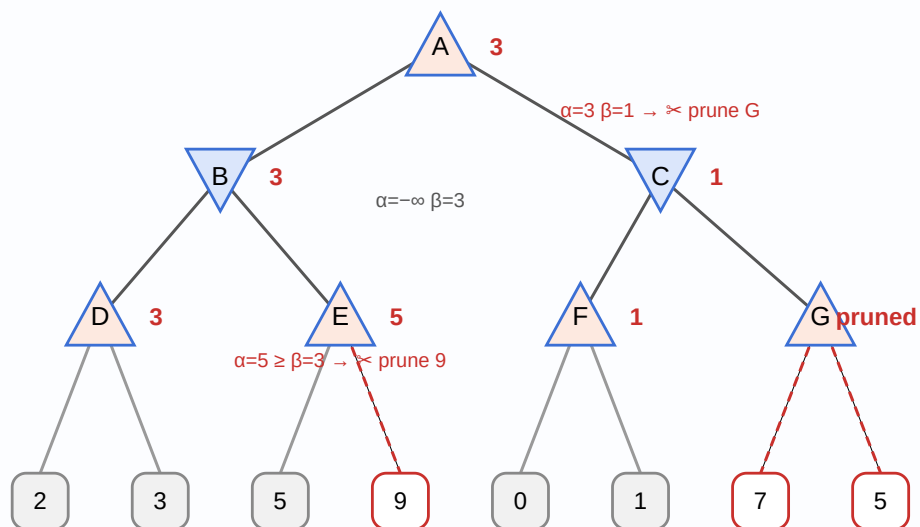


Figure B2. Alpha-beta pruning: dashed red branches are never evaluated; root value 3

Supplement C. Reasoning under uncertainty

Reasoning under uncertainty means reaching conclusions or decisions when information is incomplete, ambiguous, noisy or uncertain (a doctor cannot be sure of a disease; a weather app cannot be 100% sure of rain). **Probabilistic reasoning** uses probability theory for this: instead of "the patient definitely has the disease" the system says "there is an 85% probability".

Probability $P(A)$ = favourable outcomes / total outcomes, a value from 0 (impossible) to 1 (certain).

Example: a bag with 6 red and 4 blue balls gives $P(\text{Red}) = 6/10 = 0.6$. A diagnosis system might output $P(\text{Flu}) = 0.70$, $P(\text{COVID-19}) = 0.20$, $P(\text{Other}) = 0.10$ for the same symptoms.

Conditional probability

Probability of A given that B has occurred:

$$P(A | B) = P(A \cap B) / P(B)$$

Example: of 100 students, 60 study CS and 30 of those are female, so $P(\text{Female} | \text{CS}) = 30/60 = 0.5$.

Bayes' theorem

Updates a belief when new evidence arrives:

$$P(A | B) = P(B | A) \cdot P(A) / P(B)$$

Term	Name	Meaning
$P(A)$	Prior	Belief before evidence
$P(B A)$	Likelihood	Chance of the evidence if A is true
$P(B)$	Evidence (marginal)	Overall chance of the evidence
$P(A B)$	Posterior	Updated belief after evidence

Example. 1% of people have a disease, so $P(D) = 0.01$. The test is positive for 90% of people with the disease, $P(+|D) = 0.90$, and for 5% of people without it, $P(+|\neg D) = 0.05$.

$$P(+) = (0.90)(0.01) + (0.05)(0.99) = 0.009 + 0.0495 = 0.0585$$

$$P(D|+) = 0.009 / 0.0585 \approx \mathbf{0.154 (15.4\%)}$$

Even after a positive test the chance of disease is only about 15%, because the prior is low and false positives are common.

Conditional independence

A and B are conditionally independent given C when $P(A, B | C) = P(A | C) P(B | C)$: once C is known, B tells nothing more about A. Example: Traffic and Wet road both depend on Rain; given Rain, they can be treated as independent. This property is what makes Naïve Bayes (Q13) and Bayesian networks efficient.

Bayesian network

A **Bayesian network** is a graphical model of probabilistic relationships using a **directed acyclic graph (DAG)** with three components:

1. **Nodes** – random variables (Weather, Rain, Traffic, Disease).
2. **Directed edges** – probabilistic dependency (Rain $\rightarrow$ Wet Road).
3. **Conditional probability tables (CPTs)** – probability of a node given its parents.

Rain	P(Wet Road = Yes)
Yes	0.95
No	0.10

Medical example: Disease is the parent of Fever and Cough (child nodes), with $P(\text{Disease}) = 0.1$, $P(\text{Fever} \mid \text{Disease}) = 0.8$, $P(\text{Cough} \mid \text{Disease}) = 0.7$. The network then estimates $P(\text{Disease} \mid \text{Fever}, \text{Cough})$. The joint distribution factorises as $P(x_1, \dots, x_n) = \prod P(x_i \mid \text{parents}(x_i))$.

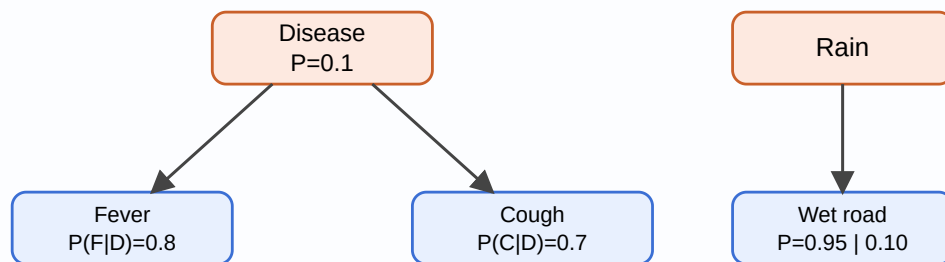


Figure C. Bayesian networks (DAGs with conditional probability tables)

Supplement D. Machine learning basics and Responsible AI

Need for machine learning

Traditional programming cannot handle complex tasks or huge data efficiently. ML learns from data without fixed rules, which helps to:

- solve complex problems (image and speech recognition, medical diagnosis, language translation);
- handle large volumes of data (fraud detection, personalised feeds);
- automate repetitive tasks (spam filtering, chatbots);
- personalise user experience (Netflix and e-commerce recommendations);
- improve itself with more data (voice assistants, search engines, self-driving cars).

How machines learn

1. **Data input** – quality and quantity of data matter.
2. **Algorithm** – mathematical method to find patterns (classification, regression).
3. **Model training** – adjust internal parameters to reduce the gap between predictions and actual results.
4. **Feedback loop** – compare predictions with true outcomes and correct errors (for example gradient descent).
5. **Experience and iteration** – repeat training to refine predictions.
6. **Evaluation and generalisation** – test on new data to confirm real-world performance.

Responsible AI

Responsible AI is designing, developing and deploying AI that is **ethical, fair, transparent and accountable**, minimising bias, risk and unintended consequences.

Principle	Meaning	Key practices
Fairness	Treat individuals and groups equally	Diverse training data, bias detection, regular monitoring
Transparency	Decisions are understandable and open	Explain decisions in loans, hiring, healthcare; interpretable, documented models
Accountability	Humans stay responsible for outcomes	Clear roles, human oversight, monitoring and governance
Privacy and security	Protect personal and sensitive data	Limit personal data use, follow privacy regulations, strong security
Reliability	Consistent, accurate and safe behaviour	Testing and validation, error analysis, resilient design
Ethical usability	Easy, inclusive and beneficial to all users	User-friendly design, clear communication, respect for user autonomy

How to implement it:

1. Define clear Responsible AI principles (with cross-functional teams).
2. Educate teams and raise awareness.
3. Integrate ethics throughout the AI lifecycle (address bias, document data and decisions).

4. Protect privacy and sensitive data.
5. Enable human oversight and accountability.
6. Monitor systems and improve continuously.